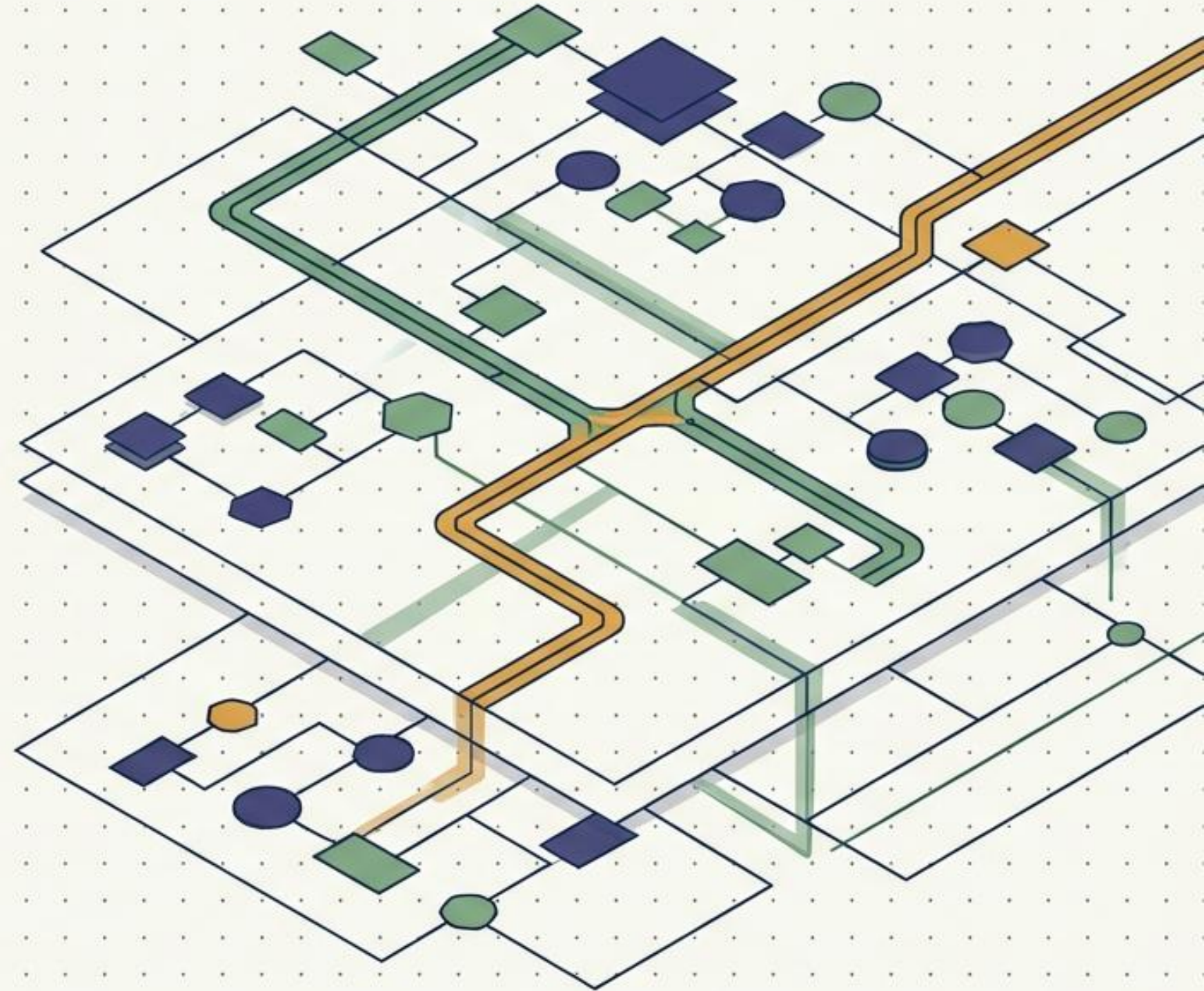


Integrating Knowledge Graphs

Architecting the
Enterprise Data Fabric

DR ZBIGNIEW GALAR



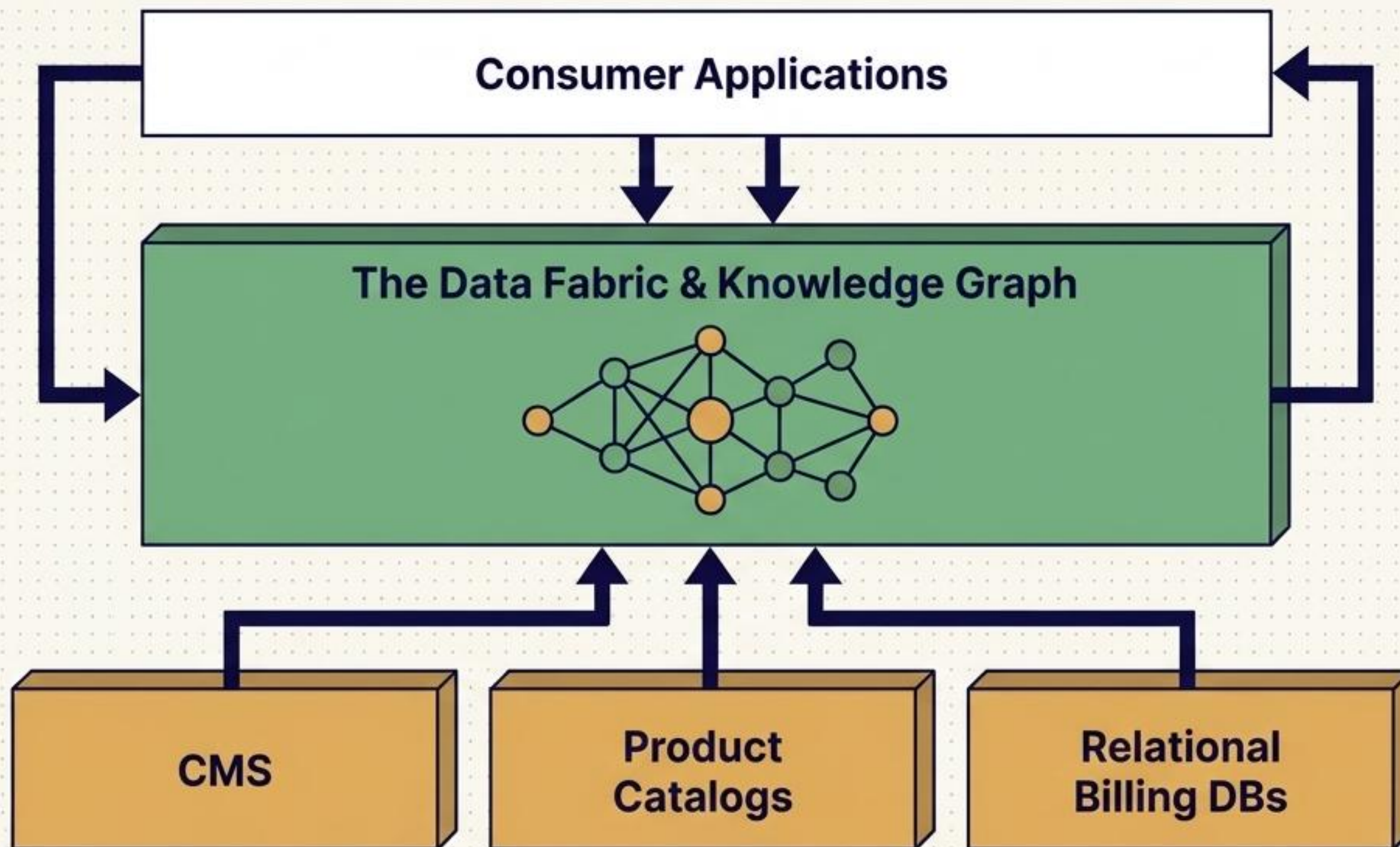
The Knowledge Graph Unlocks the Enterprise Data Fabric

The Virtuous Cycle

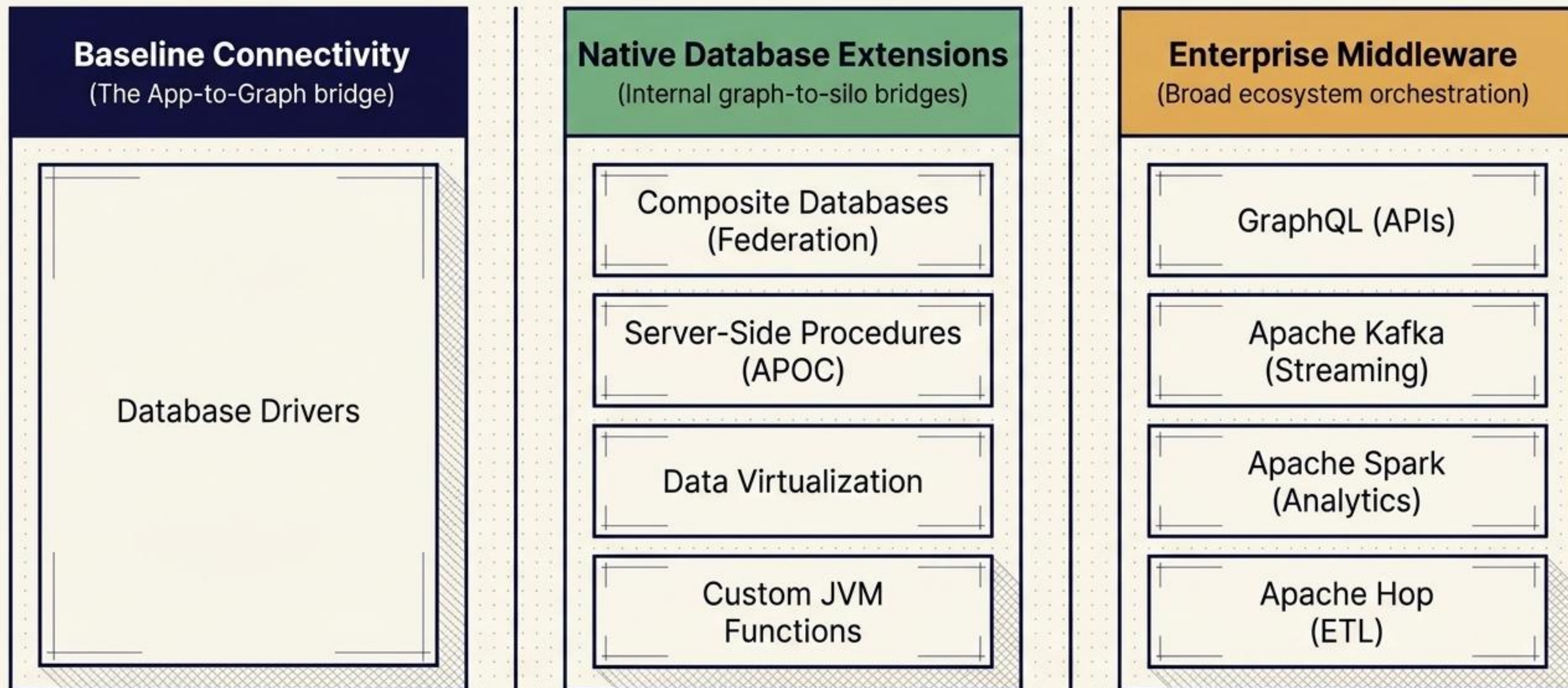
KGs enrich enterprise systems and are in turn enriched by them.

Agile Master Data

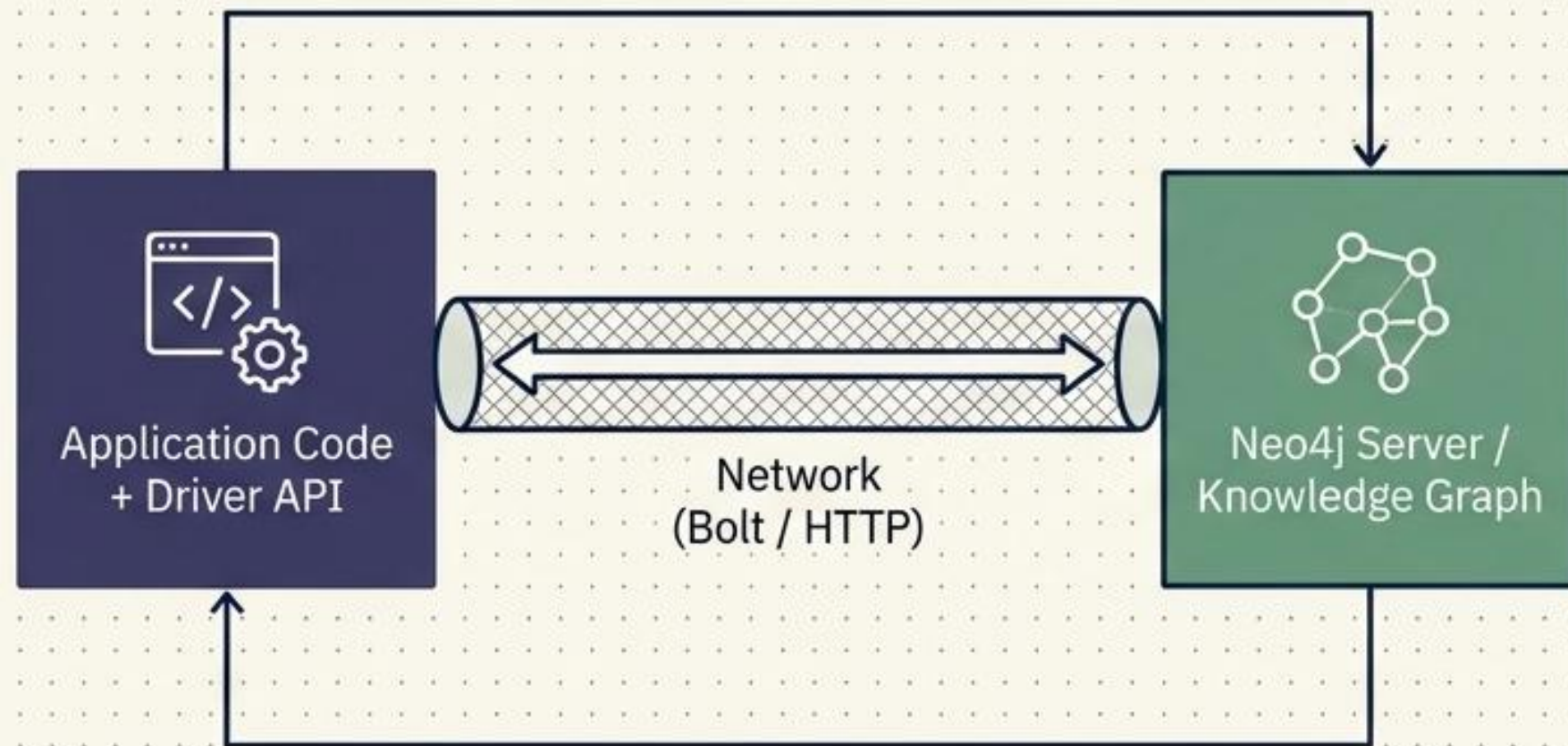
The KG acts as a sophisticated, curated **index** over physical data silos. It creates golden **records** incrementally as data is discovered, entirely avoiding the need for massive, rigid upfront design.



The Knowledge Graph Integration Landscape



Database Drivers Form the Foundational Link



The 4 Golden Rules of Drivers



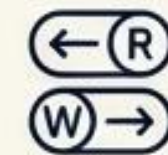
1. Drivers are expensive

Instantiate only once per application lifetime to avoid network/authentication overhead.



2. Sessions are cheap

Use them whenever needed. They support causal boundaries (bookmarks) so clients always see their own writes across distributed clusters.



3. Tag read vs. write

Explicitly routing queries optimizes cluster throughput.



4. Parameterize queries

Use parameters instead of literal strings to prevent injection attacks and utilize cached query plans.

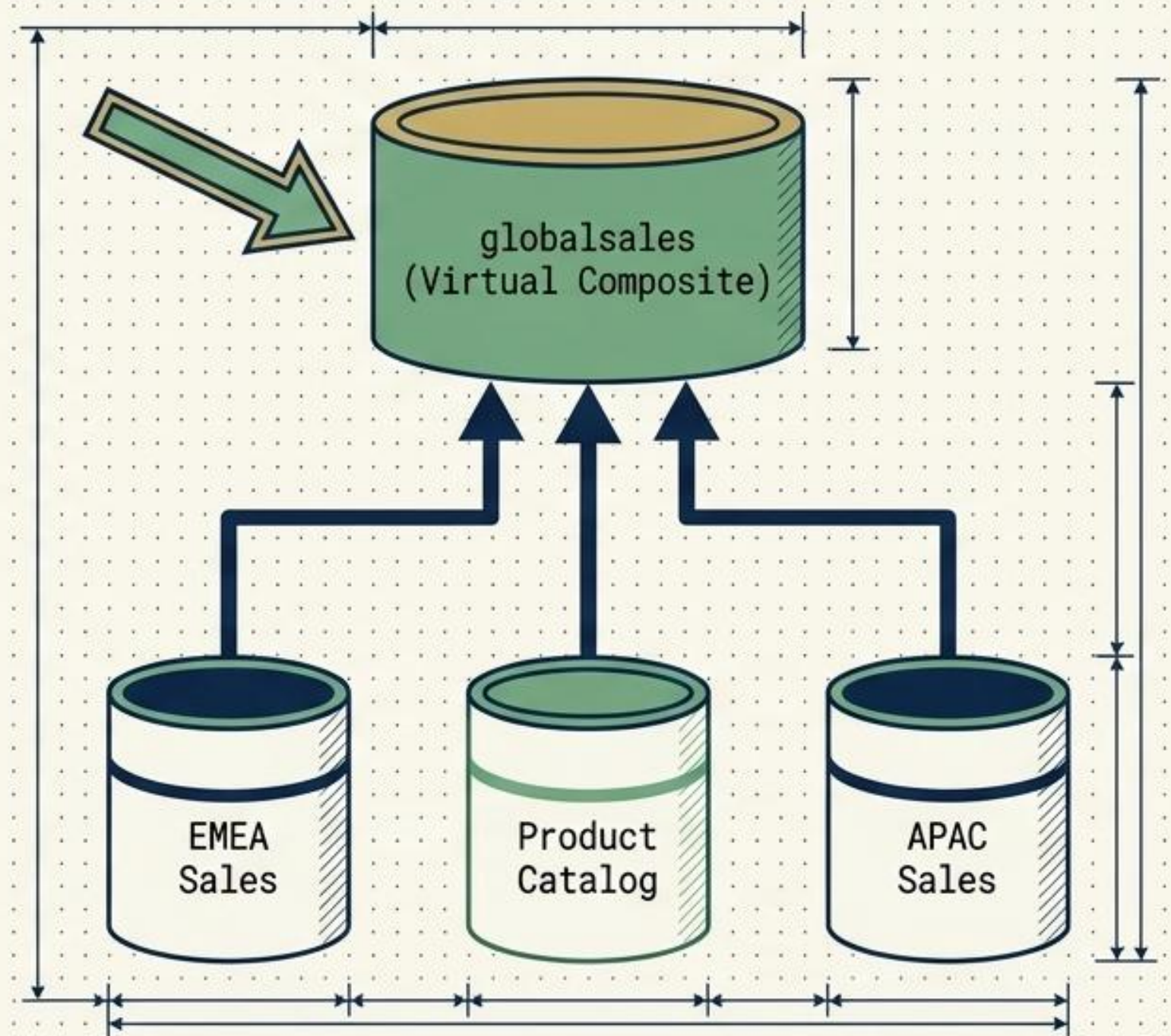
Graph Federation Creates a Unified Composite Database

The Mechanism:

The consumer application queries a single alias. Neo4j Fabric automatically routes and parallelizes the Cypher query across multiple physically separated graph databases.

The Key Advantage:

Implementation complexity is completely abstracted away from the user. Multiple independent graphs behave as one.



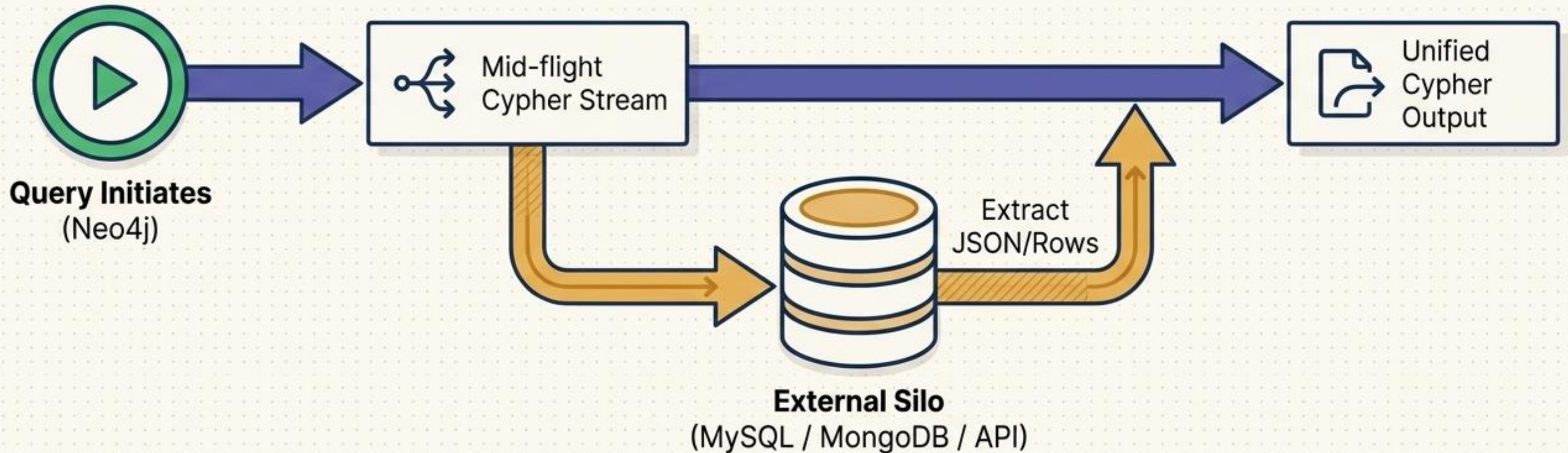
Server-Side Procedures Pull External Data at Query Time

The Concept

APOC (Awesome Procedures on Cypher) allows the KG to call external APIs or relational databases during a graph query.

Architectural Trade-off

Enrich the graph without copying or moving data from the source system. However, remote network latency directly adds to the graph query latency.



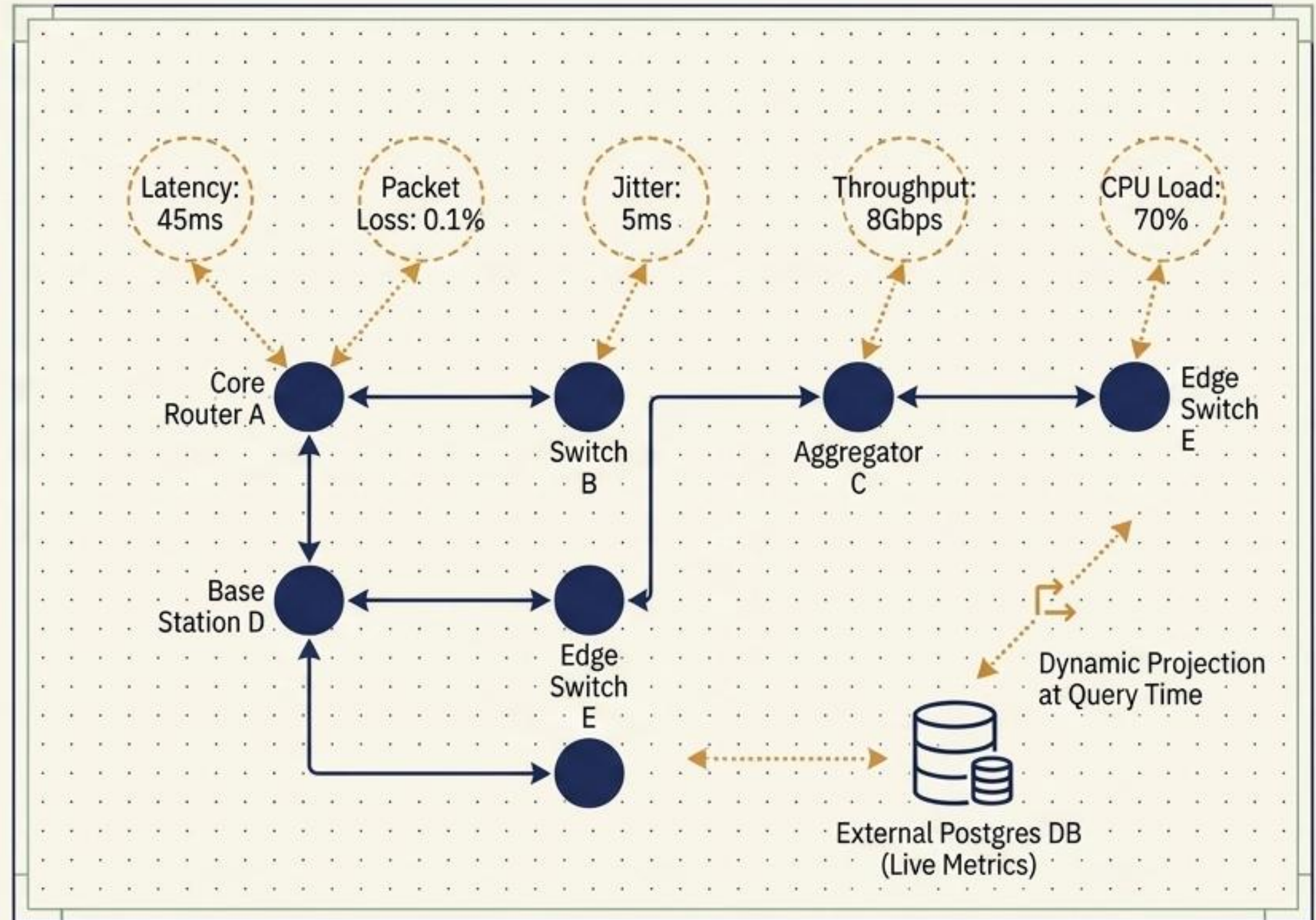
Data Virtualization Overlays Live Metrics onto Graph Topology

The Challenge:

Highly volatile, non-graph data (time-series, hardware metrics, logs) shouldn't be physically stored in a structural graph.

The Solution:

APOC defines external data sources as virtual resources. Neo4j projects SQL data as virtual nodes and relationships, seamlessly mixing them with native graph data to create live Digital Twins.



Architectural Comparison: Federation vs. Virtualization

Graph Federation (Composite DBs)	Data Virtualization (APOC)
Source Data: Multiple independent Graph databases.	Source Data: Non-Graph data (SQL, MongoDB, CSV, APIs).
Output: Merges into one virtual graph.	Output: Projects non-graph data as virtual graph nodes.
Performance: High native performance.	Performance: Subject to external database and network latency.
Best For: Sharded data, regional compliance (EMEA vs APAC), distinct domain graphs.	Best For: Fast-changing metrics, live external data enrichment without ETL copying.

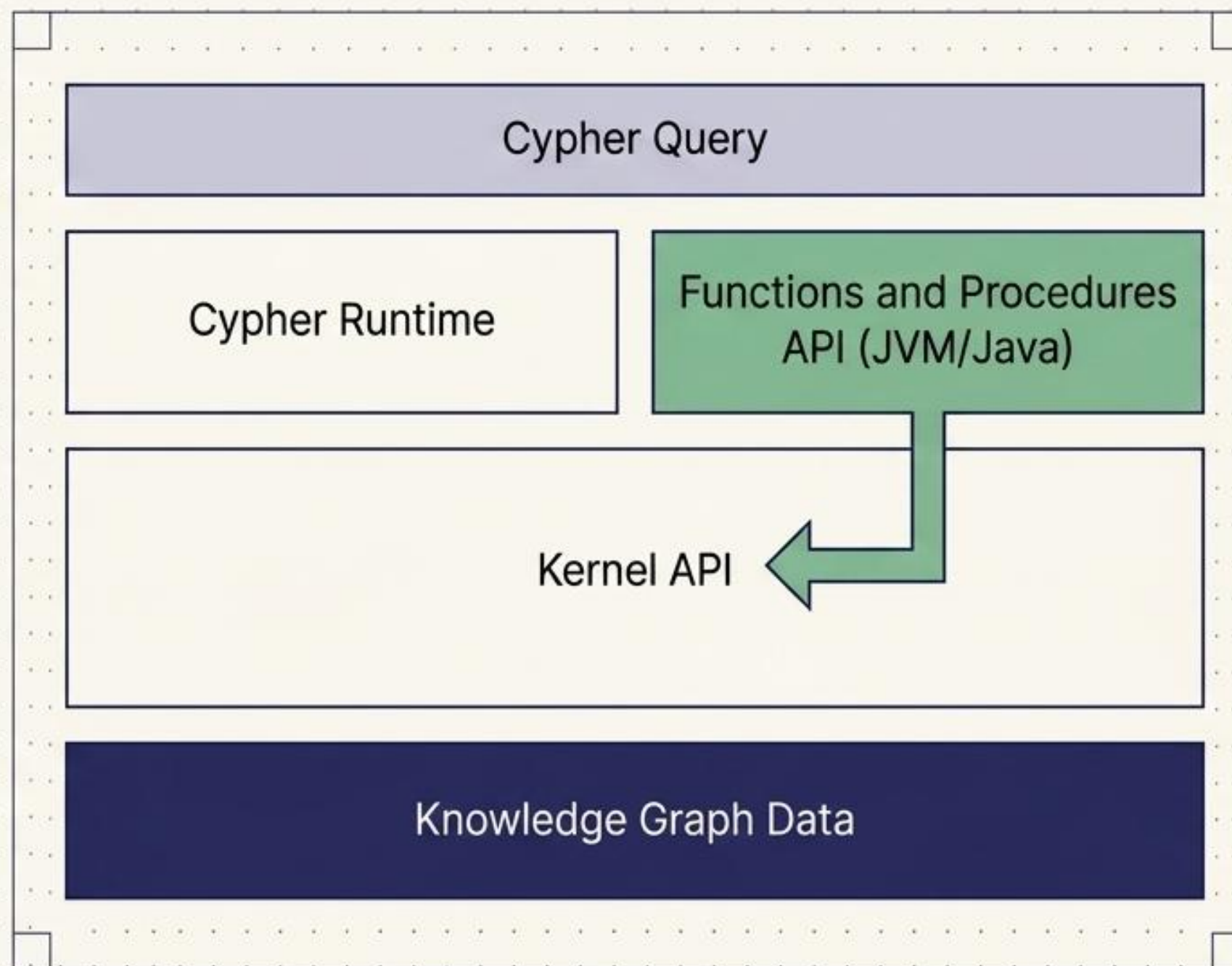
Custom JVM Functions Extend the Core Engine

The Capability

When standard Cypher and APOC aren't enough, architects can write highly specialized domain logic in JVM languages (Java, Scala).

The Modern Approach

Unlike clunky SQL stored procedures, these functions interact directly with Neo4j's Java API. They can be tested independently of the database and follow modern CI/CD software practices.



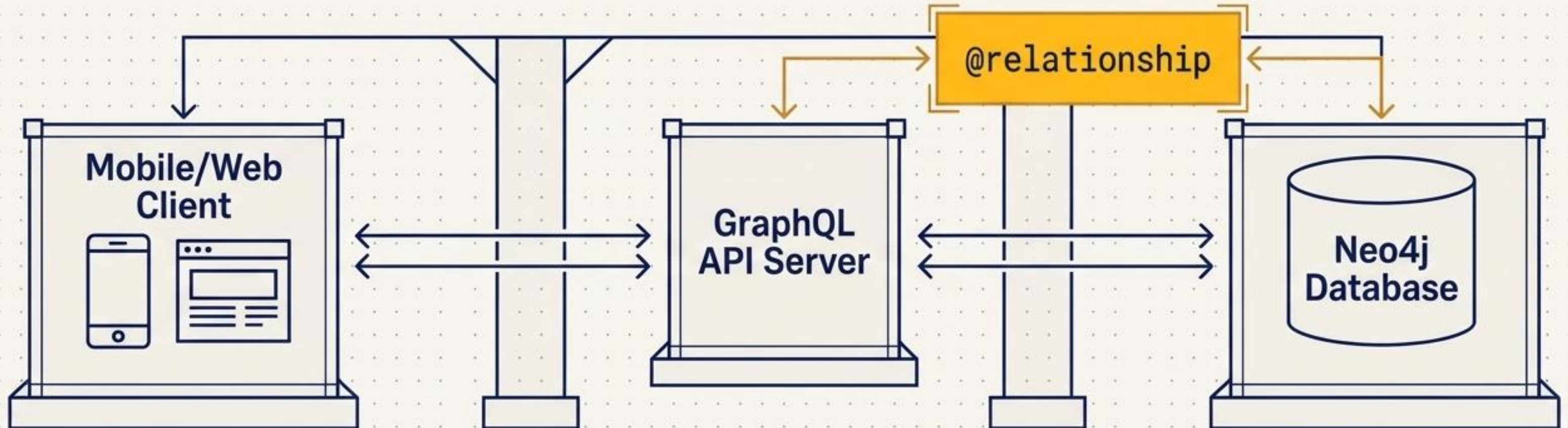
GraphQL APIs Connect the Graph to User Applications

The Bridge

GraphQL provides a strict schema and consistent graph data model for developer productivity.

The Translation

Neo4j's implementation uses the `@relationship` directive to map GraphQL objects directly to graph nodes and edges, enabling rapid API deployment.



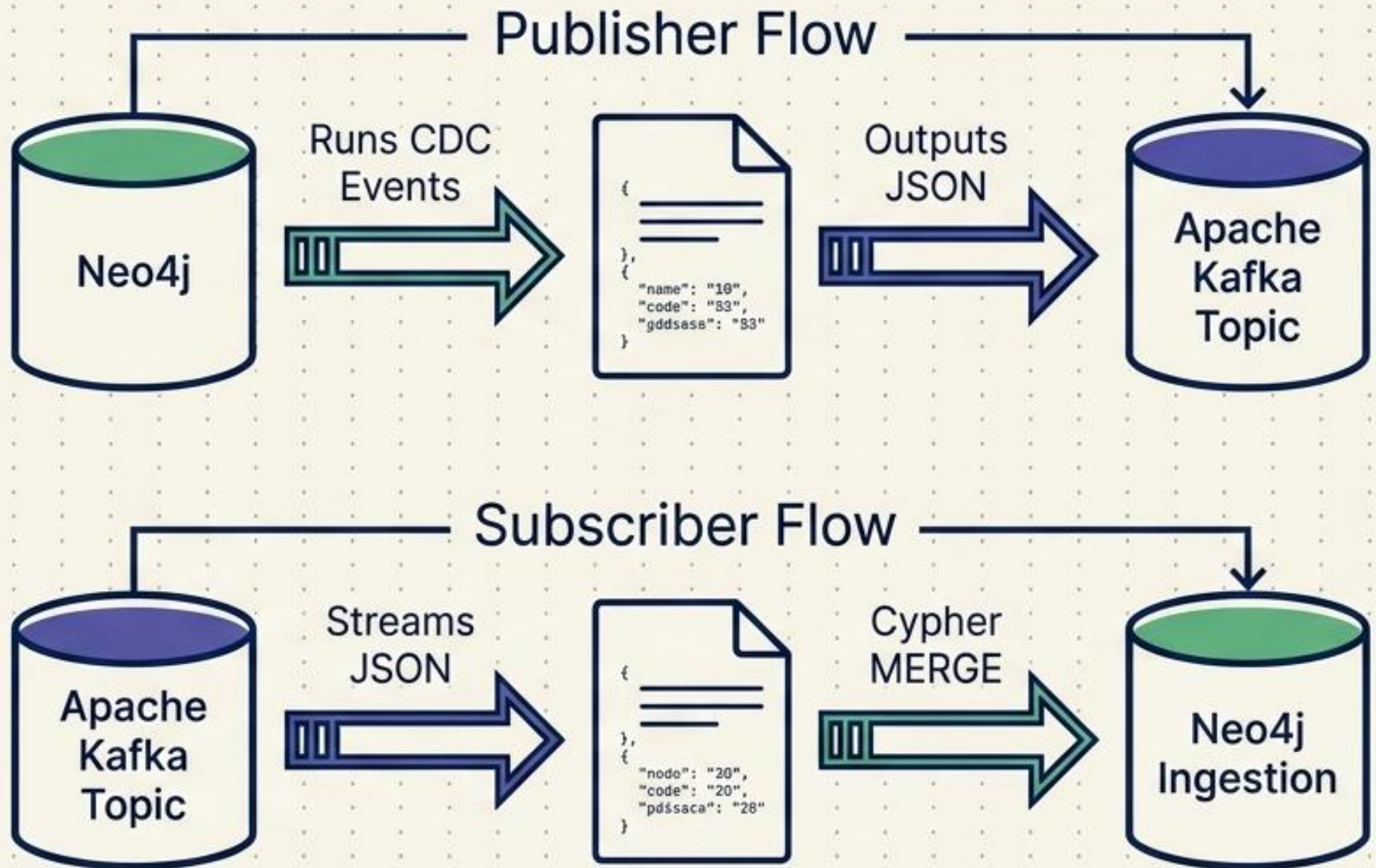
Apache Kafka Synchronizes Enterprise Events in Real-Time

The Mechanism

The Kafka Connect plug-in glues the enterprise together.

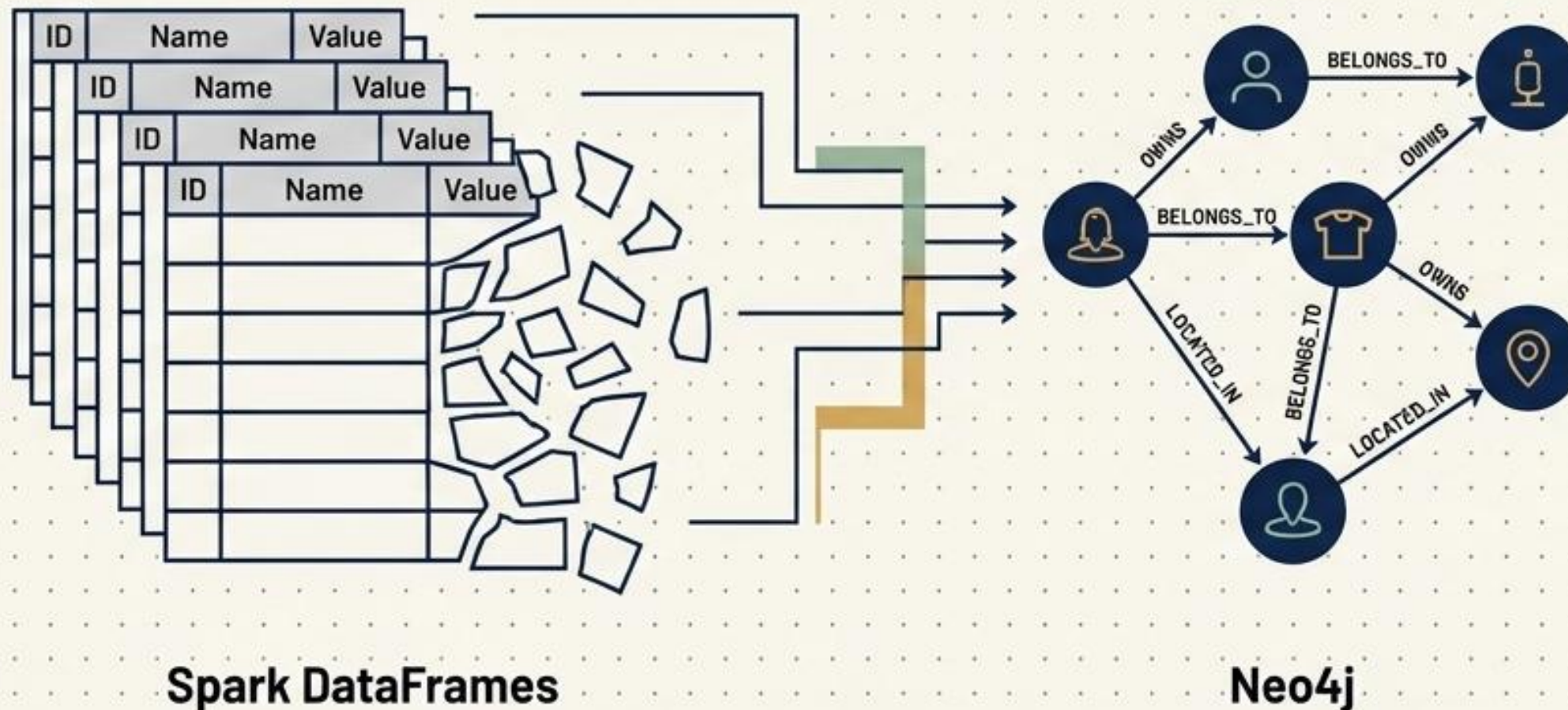
Dual Roles

The KG acts as a **Publisher** (broadcasting changes) and a **Subscriber** (ingesting high-volume external events).



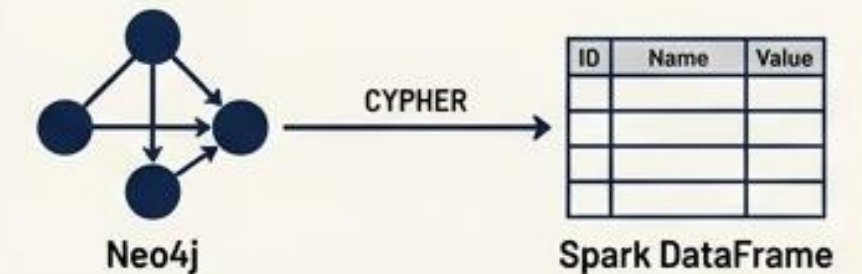
Apache Spark Orchestrates Large-Scale Analytics

The Integration: The Spark connector enables parallel big-data processing pipelines to use the KG as both a source and a sink.



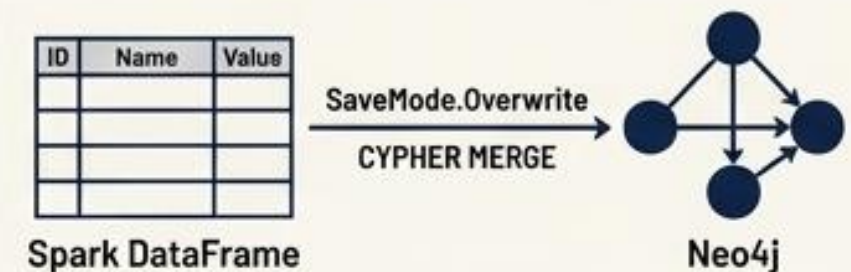
Read Mechanics

Spark pulls native graph data by label or specific Cypher query into flat DataFrames.



Write Mechanics

Spark pushes DataFrames back to the graph. To prevent data duplication, architects map Spark's SaveMode.Overwrite directly to Cypher's MERGE command.



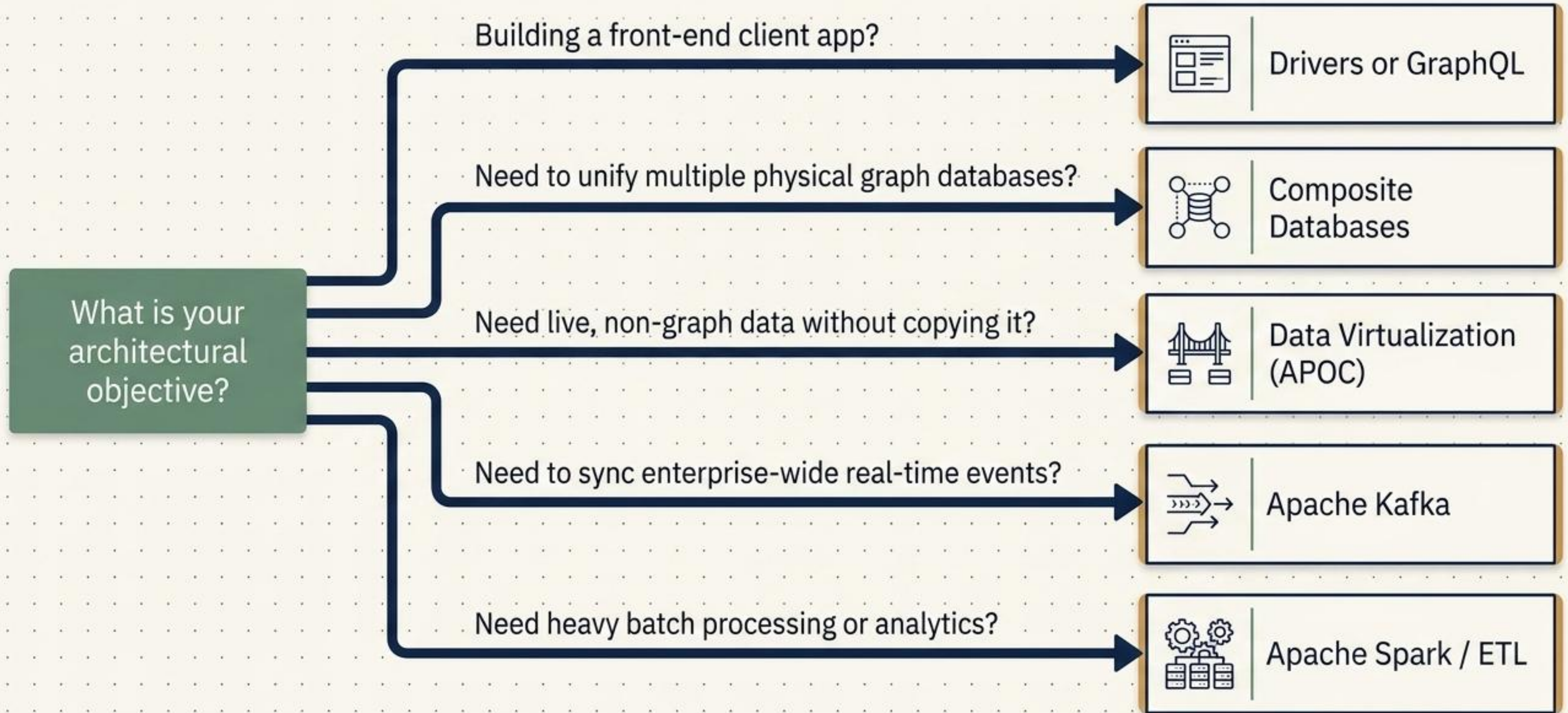
ETL Platforms Automate Low-Code Graph Ingestion

The Workflow: General-purpose ETL tools (like Apache Hop) provide visual environments to orchestrate complex data flows. Users clean data and map source fields directly to Graph Target Types without writing complex backend code.



	Source Field	Target Type	Target Property
1	'brewery'	Node	name
2	'percentage'	Relationship	pct

Diagnostic Framework: Choosing Your Integration Path



A Dependable Layer of Master Data

The Engine, Not a Store

A Knowledge Graph is an active enterprise engine, not an isolated data store.

Iterative Evolution

Flexibility is key. Because the graph model handles heterogeneous data naturally, you can integrate incrementally as data domains evolve.

The Result

An enriched, scalable, and action-ready enterprise architecture.

